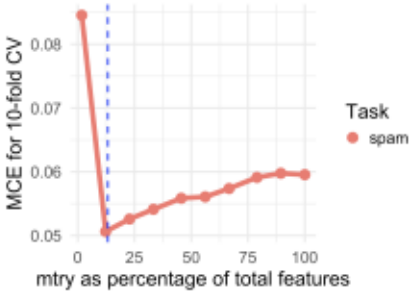
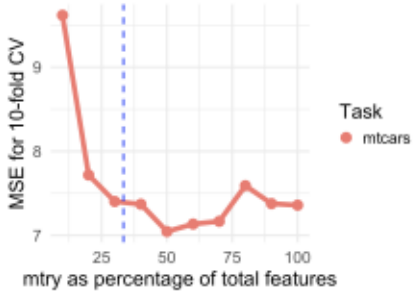
# RANDOM FEATURE SAMPLING

RFs decorrelate trees with a simple randomization:

- For each node of tree, randomly draw $\texttt{mtry} \leq p$ features ($\texttt{mtry}$ = name in some implementations)
- Only consider these features for finding the best split
- Careful: Our previous analysis was simplified! The more we decorrelate by this, the more random the trees become! This also has negative effects!

# EFFECT OF FEATURE SAMPLING



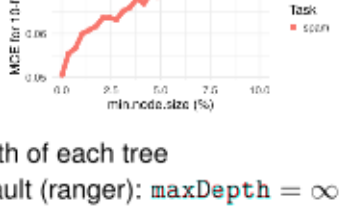



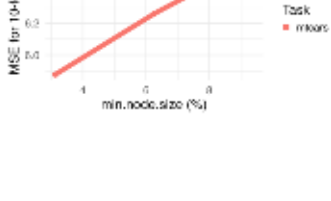
- Optimal `mtry` typically larger for regression than for classification
- Good defaults exist, but still most relevant tuning param
- Rule of thumb:
  - Classification: $\texttt{mtry} = \lfloor \sqrt{p} \rfloor$
  - Regression: $\texttt{mtry} = \lfloor p/3 \rfloor$

# TREE SIZE

In addition to `mtry`, RFs have two other important HPs:

- Min. nr. of obs. in each decision tree node
  Default (ranger): `min.node.size = 5` ▸ Breiman 2001



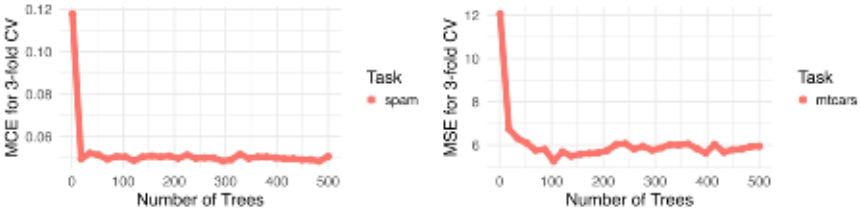




- Depth of each tree
  Default (ranger): `maxDepth` $= \infty$
- There are more alternative HPs to control depth of tree: minimal risk reduction, size of terminal nodes, etc.

RF usually use fully expanded trees, without aggressive early stopping or pruning, to further **increase variability of each tree**. ▸ Louppe 2015

# CAN RF OVERFIT? ▸ Probst and Boulesteix 2018

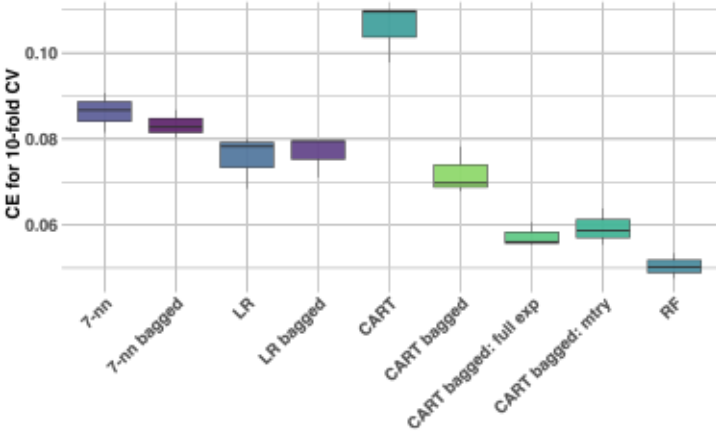
- Just like any other learner, RFs **can** overfit!
- However, RFs generally **less** prone to overfitting than individual CARTs.
- Overly complex trees can *still* lead to overfitting!
  If most trees capture noise, so does the RF.
- But randomization and averaging helps.




Since each tree is trained *individually and without knowledge of previously trained trees*, increasing `ntrees` generally reduces variance **without increasing the chance of overfitting!**

# RF IN PRACTICE

Benchmarking bagged ensembles with 100 BLs each on `spam` versus RF ($\texttt{ntrees} = 100$, $\texttt{mtry} = \sqrt{p}$, $\texttt{minnode} = 1$), we see how well RF performs!




$\Rightarrow$ RFs combine the benefits of random feature selection and fully expanded trees.